

Proyecto Fundamentos Básicos de Python

Maria Isabela Hernandez Muñoz
Docente: Alejandro Rodas Vásquez
Universidad Tecnológica de Pereira

29 de septiembre de 2026

Introducción

En este proyecto usted pondrá en práctica los conceptos básicos vistos en esta unidad especialmente *arreglos*, *funciones*, *módulos*. Se deberá consumir los recursos suministrados por una API suministrada por el portal Nacional de [Datos Abiertos](#) y crear una Arquitectura de Software basada en módulos.

Durante esta pandemia la adquisición, almacenamiento y consulta de datos relacionado con el reporte de casos de COVID-19 se ha convertido en un insumo imprescindible para la creación de aplicaciones software que ayuden en la prevención y trazabilidad de la enfermedad.

El portal de Datos Abierto pone a disposición el conjunto de datos llamado [Casos positivos de COVID-19 en Colombia](#) (se recomienda que sea examinado) el cual puede ser consultado por aplicaciones web o de escritorio a través de consultas a su API. A continuación, se encuentra el código fuente en Python que permite realiza dicha consulta.

```

1 import pandas as pd
2 from sodapy import Socrata
3
4 # Unauthenticated client only works with public data sets. Note 'None'
5 # in place of application token, and no username or password:
6 client = Socrata("www.datos.gov.co", None)
7
8 # Example authenticated client (needed for non-public datasets):
9 # client = Socrata(www.datos.gov.co,
10 #                  MyAppToken,
11 #                  username="user@example.com",
12 #                  password="AFakePassword")
13
14 # First 2000 results, returned as JSON from API / converted to Python
15 # list of dictionaries by sodapy.
16 results = client.get("gt2j-8ykr", limit=limite_registros, departamento =
17                     nombre_departamento)
18
19 # Convert to pandas DataFrame
20 results_df = pd.DataFrame.from_records(results)

```

En las líneas 1 y 2 se hace la llamada a las librerías *Pandas* y *sodapy*. *Pandas* es reconocida por ser empleada en *Análisis de Datos*. En la línea 16 se encuentran los parámetros *limite_registros* y *nombre_departamento*. Estos parámetros son requeridos y como su nombre lo dice, permiten consultar el Departamento y el Número de Casos que se quieren extraer. Para mayor conocimiento sobre el API se tiene el sitio [Documentación sodapy](#).

1. Requerimientos Funcionales

Usted debe de crear una aplicación que le permita al usuario ingresar el Departamento (*nombre_departamento*) que desea consultar y el Número de Registro (*limite_registros*) que quiere obtener de la consulta (este parámetro es importante puesto que si selecciona un número grande e.j 1000 registros, la consulta puede “colgarse”).

El resultado obtenido debe de poder visualizarse en la pantalla en un formato (investigar la función *format*) que contenga solo las siguientes columnas: *Ciudad de ubicación, Departamento, Edad, Tipo, Estado y País de procedencia*.

2. Requerimientos de la Arquitectura de Software

El software debe de estar construido aplicando el concepto de *modularidad*. Cada módulo debe de tener una tarea en específico

- Módulo para UI
- Módulo API
- Como archivo separado el *main*

3. Evidencias para la entrega

1. Usted debe de subir el código fuente a su repositorio invidual de [github](#).
2. Pantallazos donde se corrobore el funcionamiento del software (consultas realizadas y resultado obtenido) y se compruebe que los Requerimientos Funcionales ([Sección 1](#)) han sido cumplidos.
3. Para que esta actividad esté enmarcada dentro del contexto de la Arquitectura de Software y tengo un fundamento teórico válido. Usted debe de consultar ¿Qué es un Diagrama de Componentes? ¿Para qué se utiliza? ¿Qué es un componente?

3.1. Estructura del Proyecto

A continuación, se muestra la arquitectura que debe de tener el software. Sin embargo, en esta estructura solo se muestra la base, se debe agregar los archivos Python que usted crea.

```
ProyectoAPI/  
|-- api/  
|   |-- __init__.py  
|   '-- consulta_api.py  
|-- ui/  
|   |-- __init__.py  
|   '-- interfaz.py  
|-- main.py  
|-- requirements.txt  
'-- README.md
```

3.2. Código fuente

main.py Punto de entrada: coordina la UI con la API.

```
1 """Punto de entrada de la aplicación.  
2 Coordina el módulo ui (entrada/salida) con el módulo api (datos).  
3 """  
4 from api import consultar_casos  
5 from ui import pedir_datos, mostrar_resultados, mostrar_mensaje  
6  
7  
8 def main():  
9     while True:  
10         departamento, limite = pedir_datos()  
11         mostrar_mensaje("\nConsultando la API, espere un momento...")  
12         try:  
13             df = consultar_casos(departamento, limite)  
14         except ConnectionError as error:  
15             mostrar_mensaje(f"\nError: {error}")  
16         else:  
17             mostrar_resultados(df)  
18
```

```

19         otra = input("\n¿Desea hacer otra consulta? (s/n):
20         ").strip().lower()
21         if otra != "s":
22             mostrar_mensaje("Hasta luego.")
23             break
24
25 if __name__ == "__main__":
26     main()

```

api/consulta_api.py Módulo API: consulta y formatea los datos.

```

1  """Consulta a la API de Datos Abiertos (Socrata) del conjunto
2  'Casos positivos de COVID-19 en Colombia' (id: gt2j-8ykr).
3
4  Este módulo solo se encarga de OBTENER y FILTRAR los datos.
5  No imprime nada ni pide datos al usuario (eso es tarea del módulo ui).
6  """
7  import pandas as pd
8  from sodapy import Socrata
9
10 DOMINIO = "www.datos.gov.co"
11 ID_DATASET = "gt2j-8ykr"
12 TIMEOUT_SEGUNDOS = 60
13
14 # Nombre que se muestra al usuario -> posibles columnas reales del
15 # dataset.
16 # Se listan varias porque el portal ha cambiado el esquema con el tiempo.
17 COLUMNAS_SALIDA = {
18     "Ciudad de ubicación": ["ciudad_municipio_nom",
19                             "ciudad_de_ubicaci_n"],
20     "Departamento": ["departamento_nom", "departamento"],
21     "Edad": ["edad"],
22     "Tipo": ["fuente_tipo_contagio", "tipo"],
23     "Estado": ["estado"],
24     "País de procedencia": ["pais_viajo_1_nom", "pais_de_procedencia"],
25 }
26
27 # Columnas por las que se puede filtrar el departamento (se prueban en
28 # orden).
29 COLUMNAS_FILTRO_DEPARTAMENTO = ["departamento_nom", "departamento"]
30
31 def _crear_cliente():
32     """Cliente sin autenticación (el dataset es público)."""
33     return Socrata(DOMINIO, None, timeout=TIMEOUT_SEGUNDOS)
34
35 def _primera_columna_existente(df, candidatas):
36     for nombre in candidatas:
37         if nombre in df.columns:
38             return nombre
39     return None

```

```

39
40
41 def _formatear(df):
42     """Deja solo las columnas requeridas, con sus nombres legibles."""
43     salida = pd.DataFrame()
44     for titulo, candidatas in COLUMNAS_SALIDA.items():
45         col = _primera_columna_existente(df, candidatas)
46         salida[titulo] = df[col] if col else "N/D"
47     return salida.fillna("N/D")
48
49
50 def consultar_casos(nombre_departamento, limite_registros):
51     """Devuelve un DataFrame con los casos del departamento indicado.
52
53     Parámetros:
54         nombre_departamento (str): p. ej. "Risaralda".
55         limite_registros (int): cantidad máxima de registros a traer.
56     """
57     client = _crear_cliente()
58     variantes = [nombre_departamento.strip().upper(),
59                  nombre_departamento.strip()]
60
61     ultimo_error = None
62     for columna in COLUMNAS_FILTRO_DEPARTAMENTO:
63         for valor in variantes:
64             try:
65                 # Equivale a: client.get("gt2j-8ykr", limit=...,
66                 # departamento=...)
67                 results = client.get(
68                     ID_DATASET, limit=limite_registros, **{columna: valor}
69                 )
70             except Exception as error: # columna inexistente, red,
71                 timeout...
72                 ultimo_error = error
73                 continue
74             if results:
75                 df = pd.DataFrame.from_records(results)
76                 return _formatear(df)
77
78     if ultimo_error is not None:
79         raise ConnectionError(f"No se pudo consultar la API:
80 {ultimo_error}")
81     return pd.DataFrame(columns=list(COLUMNAS_SALIDA))

```

ui/interfaz.py Módulo UI: entrada y salida por consola, con la función *format*.

```

1 """Interfaz de usuario por consola.
2 Solo pide datos y muestra resultados; NO consulta la API.
3 """
4 LIMITE_MAXIMO = 500 # evita que la consulta se "cuelgue"
5
6 ENCABEZADO = "{:<22} {:<16} {:>5} {:<20} {:<12} {:<20}"
7 FILA = "{:<22.22} {:<16.16} {:>5} {:<20.20} {:<12.12} {:<20.20}"

```

```

8
9
10 def mostrar_mensaje(texto):
11     print(texto)
12
13
14 def pedir_datos():
15     """Pide departamento y número de registros. Devuelve (str, int)."""
16     print("=" * 60)
17     print(" CASOS POSITIVOS DE COVID-19 EN COLOMBIA (datos.gov.co)")
18     print("=" * 60)
19
20     while True:
21         departamento = input("Departamento a consultar (ej. Risaralda):
22 ").strip()
23         if departamento:
24             break
25         print(" El departamento no puede estar vacío.")
26
27     while True:
28         texto = input(f"Número de registros (1-{LIMITE_MAXIMO}):
29 ").strip()
30         if texto.isdigit() and 1 <= int(texto) <= LIMITE_MAXIMO:
31             return departamento, int(texto)
32         print(f" Ingrese un número entero entre 1 y {LIMITE_MAXIMO}.")
33
34 def mostrar_resultados(df):
35     """Imprime el DataFrame con formato de tabla usando str.format()."""
36     if df.empty:
37         print("\nNo se encontraron registros para esa consulta.")
38         return
39
40     encabezado = ENCABEZADO.format(*df.columns)
41     print("\n" + encabezado)
42     print("-" * len(encabezado))
43     for fila in df.itertuples(index=False):
44         print(FILA.format(*[str(dato) for dato in fila]))
45     print("-" * len(encabezado))
46     print(f"Total de registros mostrados: {len(df)}")

```

3.3. Pantallazos

4. Fundamento teórico: Diagrama de Componentes

4.1. ¿Qué es un componente?

Un componente es una parte modular, reemplazable e independiente de un sistema que encapsula su contenido (código y datos) y expone su funcionalidad mediante interfaces bien definidas. Otros componentes lo usan sin necesitar conocer su implementación interna. En este proyecto, *ui*, *api* y *main* son componentes.

```

PS C:\Users\herna\OneDrive\Documentos\UTP\2026-2\Programacion_3\Proyectos\ProyectoAPI1\ProyectoAPI> python .\main.py
=====
CASOS POSITIVOS DE COVID-19 EN COLOMBIA (datos.gov.co)
=====
Departamento a consultar (ej. Risaralda): Risaralda
Número de registros (1-500): 12

Consultando la API, espere un momento...
WARNING:root:Requests made without an app_token will be subject to strict throttling limits.

```

Ciudad de ubicación	Departamento	Edad	Tipo	Estado	País de procedencia
BELEN DE UMBRIA	RISARALDA	81	Comunitaria	Leve	N/D
SANTA ROSA DE CABAL	RISARALDA	64	Comunitaria	Leve	N/D
SANTA ROSA DE CABAL	RISARALDA	40	Comunitaria	Leve	N/D
PEREIRA	RISARALDA	20	Comunitaria	Leve	N/D
SANTA ROSA DE CABAL	RISARALDA	20	Comunitaria	Leve	N/D
PEREIRA	RISARALDA	49	Comunitaria	Leve	N/D
DOSQUEBRADAS	RISARALDA	34	Comunitaria	Leve	N/D
PEREIRA	RISARALDA	17	Comunitaria	Leve	N/D
PEREIRA	RISARALDA	46	Comunitaria	Leve	N/D
PEREIRA	RISARALDA	47	Comunitaria	Leve	N/D
DOSQUEBRADAS	RISARALDA	64	Comunitaria	Leve	N/D
DOSQUEBRADAS	RISARALDA	39	Comunitaria	Leve	N/D

```

-----
Total de registros mostrados: 12

```

Figura 1: Consulta 1

```

¿Desea hacer otra consulta? (s/n): s
=====
CASOS POSITIVOS DE COVID-19 EN COLOMBIA (datos.gov.co)
=====
Departamento a consultar (ej. Risaralda): Antioquia
Número de registros (1-500): 23

Consultando la API, espere un momento...
WARNING:root:Requests made without an app_token will be subject to strict throttling limits.

```

Ciudad de ubicación	Departamento	Edad	Tipo	Estado	País de procedencia
ENVIGADO	ANTIOQUIA	37	Comunitaria	Leve	N/D
ENVIGADO	ANTIOQUIA	37	Comunitaria	Leve	N/D
ITAGUI	ANTIOQUIA	36	Comunitaria	Leve	N/D
ENVIGADO	ANTIOQUIA	63	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	52	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	22	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	42	Comunitaria	Leve	N/D
BELLO	ANTIOQUIA	40	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	52	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	60	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	25	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	33	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	25	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	35	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	29	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	70	Comunitaria	Leve	N/D
COPACABANA	ANTIOQUIA	34	Comunitaria	Leve	N/D
ITAGUI	ANTIOQUIA	48	Relacionado	Leve	N/D
ITAGUI	ANTIOQUIA	60	Comunitaria	Leve	N/D
ITAGUI	ANTIOQUIA	32	Relacionado	Leve	N/D
MEDELLIN	ANTIOQUIA	34	Comunitaria	Leve	N/D
MEDELLIN	ANTIOQUIA	60	Relacionado	Leve	N/D
MEDELLIN	ANTIOQUIA	64	Comunitaria	Leve	N/D

```

-----
Total de registros mostrados: 23

¿Desea hacer otra consulta? (s/n): n
Hasta luego.
PS C:\Users\herna\OneDrive\Documentos\UTP\2026-2\Programacion_3\Proyectos\ProyectoAPI1\ProyectoAPI> |

```

Figura 2: Consulta 2

4.2. ¿Qué es un Diagrama de Componentes?

Es un diagrama estructural de UML que muestra cómo se organiza un sistema en componentes y cuáles son las dependencias e interfaces entre ellos.

4.3. ¿Para qué se utiliza?

Sirve para visualizar la arquitectura de alto nivel, dividir responsabilidades, planificar el trabajo en equipo, facilitar el mantenimiento (se puede cambiar un componente sin afectar a los demás) y comunicar el diseño a otras personas.

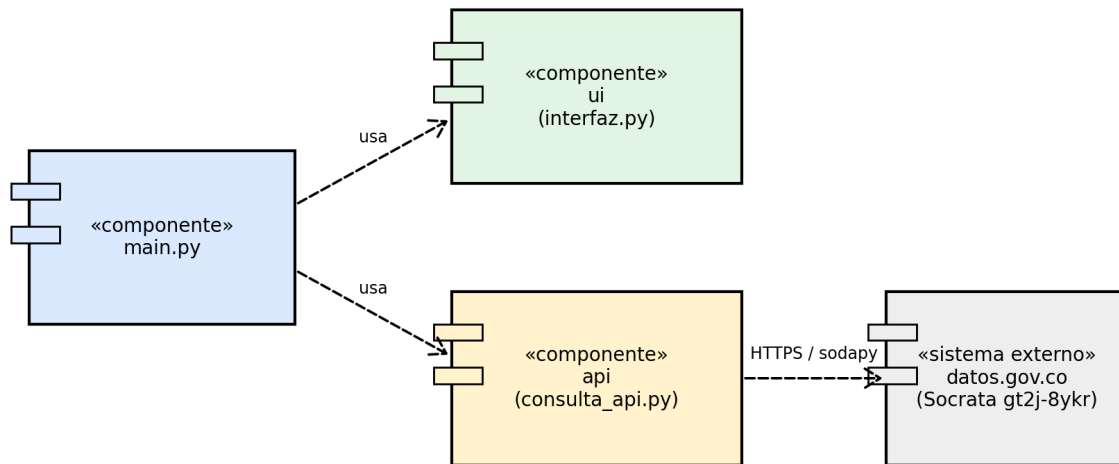


Figura 3: Diagrama de componentes del proyecto

Descripción: *main* depende de *ui* (pide los datos y muestra el resultado) y de *api* (consulta los casos). El componente *api* se comunica con el sistema externo *datos.gov.co* a través de la librería *sodapy*. *ui* y *api* no se conocen entre sí, lo que reduce el acoplamiento.